PROGRESS LOG

PT100 Thermometer

3-wire PT100 sensor → Arduino Uno → 16x2 LCD, from breadboard to 3D-printed case

Nethra Gupta
ee26btech11219

Sara Tantarpale
ee26btech11227

This log records what we built, every mistake we made along the way, and how each one was found and fixed.

01 Components We Used

Component	Value / notes
Arduino Uno	Microcontroller with a 10-bit ADC (0–1023)
PT100 (3-wire)	Platinum RTD: 100 Ω at 0 °C, rises about 0.39 Ω per °C. Two white wires, one red wire
16x2 LCD (JHD 162A)	Used in 4-bit mode
2.2 kΩ resistor	Fixed resistor in our first (initial) circuit
2 × 4.7 kΩ resistors	Added later as the top resistors of the two voltage dividers. Measured: 4650 Ω and 4600 Ω
2 × 220 Ω resistors	In parallel as the reference resistor. Measured: 238 Ω and 237 Ω (118.75 Ω together)
1 MΩ potentiometer	LCD contrast
Breadboard, jumper wires, multimeter	Prototyping and checking values
Perfboard, header pins, solder	Final soldered version

02 Testing the LCD

Before adding the sensor we wired the LCD alone in 4-bit mode and checked that it displayed text. The code uses `LiquidCrystal lcd(7, 8, 9, 10, 11, 12);`.

LCD pin	Name	Connected to
1	VSS	GND
2	VDD	5V
3	VO (contrast)	Potentiometer middle pin
4	RS	D7
5	RW	GND
6	E	D8
7–10	D0–D3	Not connected (4-bit mode)
11–14	D4–D7	D9, D10, D11, D12
15	LED+ (backlight A)	5V (the JHD 162A has a built-in backlight resistor)
16	LED− (backlight K)	GND

The potentiometer's outer legs go to 5V and GND and its middle leg goes to pin 3. A 1 MΩ pot works but makes contrast hard to set; 10 kΩ is the usual value.

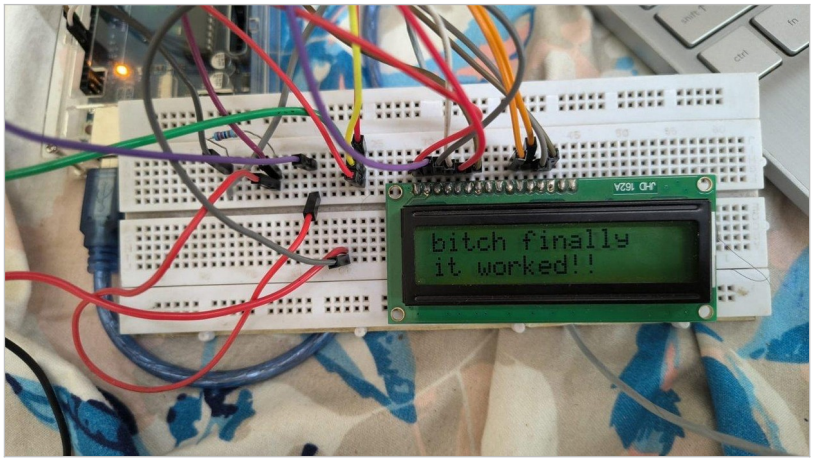


Figure 1. *Initial LCD check: our test message displayed correctly*

03 Building the Circuit

Initial connections

Our first circuit was a single voltage divider with a **2.2 kΩ fixed resistor** and the PT100, powered from the Arduino's 5V:

- 5V → 2.2 kΩ → A0 junction → PT100 white wire, PT100 red wire → GND. The second white wire was left unconnected.
- The LCD and contrast pot were wired as in the LCD check above.

Because the divider and the ADC both run from the same 5V, the PT100 resistance follows directly from the ADC reading: R_PT100 = 2200 × counts / (1024 − counts).

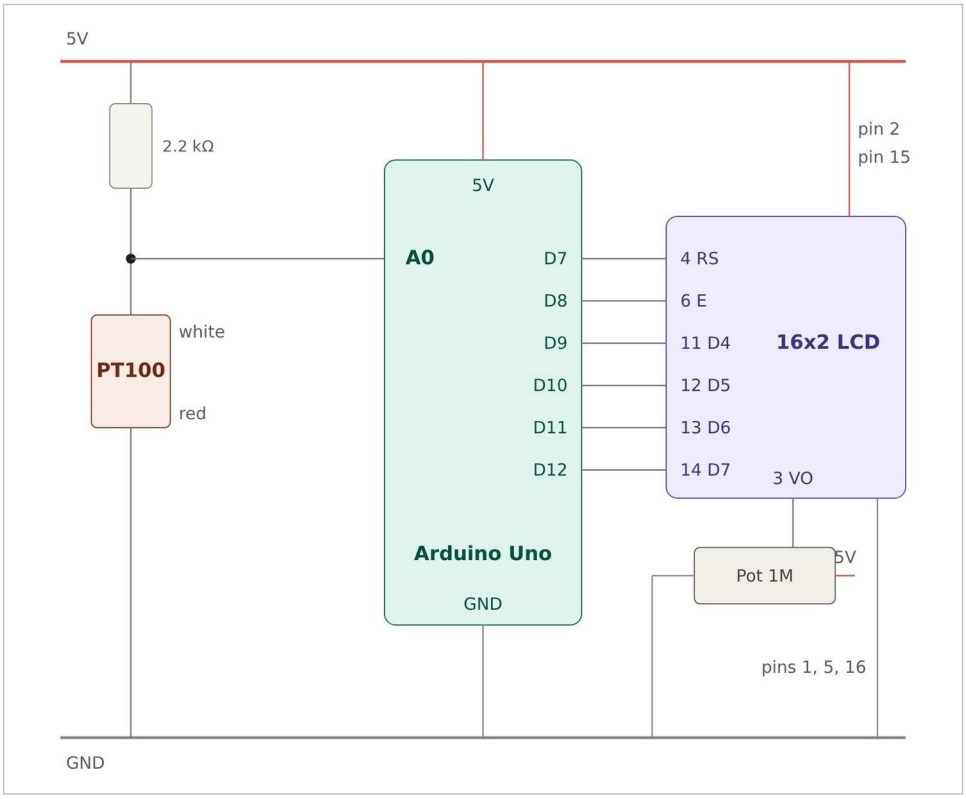


Figure 2. *Initial circuit: 2.2 kΩ fixed resistor and PT100*

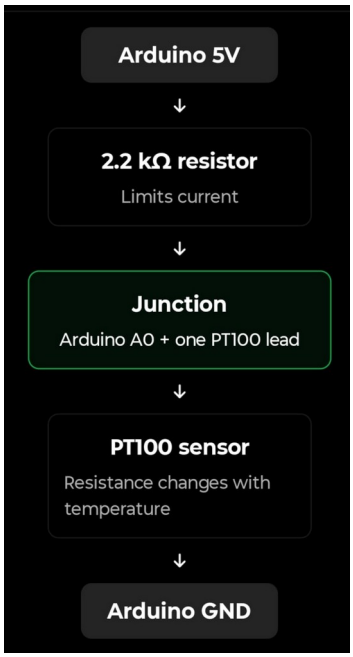


Figure 3. *How the initial circuit is connected, from 5V through the 2.2 kΩ resistor and the PT100 to GND*

Getting highly inaccurate readings

With this circuit and our first sketch, the LCD showed about **40 °C** when the actual reading was about 30 °C.

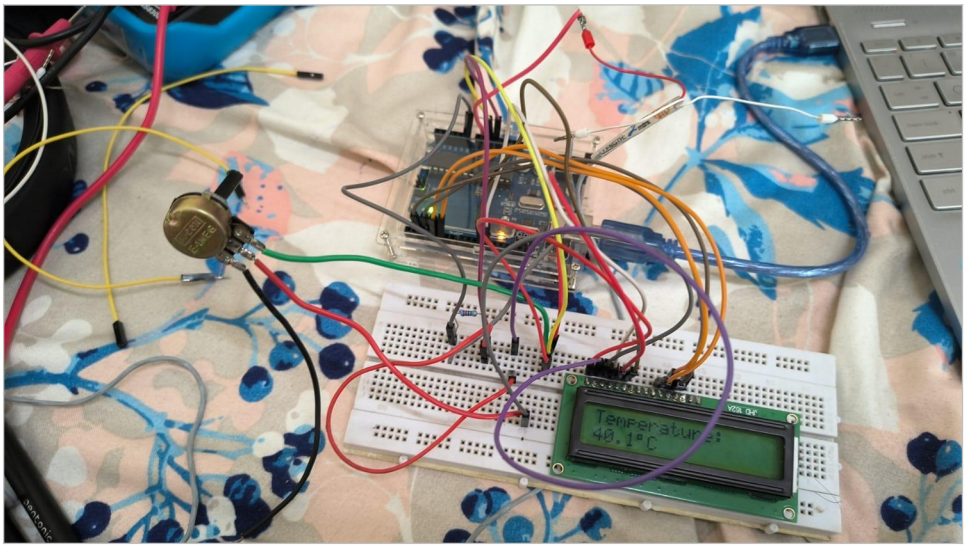


Figure 4. *First circuit with the 2.2 kΩ resistor: 40.1 °C*

Problems with the initial circuit:

1. **The signal is small compared with the ADC step.** With 2.2 kΩ on top, A0 sits at only about 0.24 V and changes by about 0.8 mV per °C. On the default 5 V reference one ADC step is 4.9 mV, which is about **°C per step**, so being off by a step or two gives a reading like 40 °C.
2. **More current than a PT100 should have.** About **2.2 mA** flows through the sensor (about 0.5 mW). PT100s are meant to be run at about 1 mA so they don't warm themselves up.
3. **No way to check the supply.** The calculation assumed the 5V rail and the 2.2 kΩ resistor were exact, with nothing in the circuit to correct for either.

New connections: added two 4.7 kΩ resistors

We rebuilt the sensor part as **two voltage dividers**, each with a 4.7 kΩ resistor on top:

- **PT100 divider (A0):** 5V → 4.7 kΩ → A0 junction → PT100 white wire, PT100 red wire → GND.
- **Reference divider (A1):** 5V → 4.7 kΩ → A1 junction → two 220 Ω in parallel → GND.

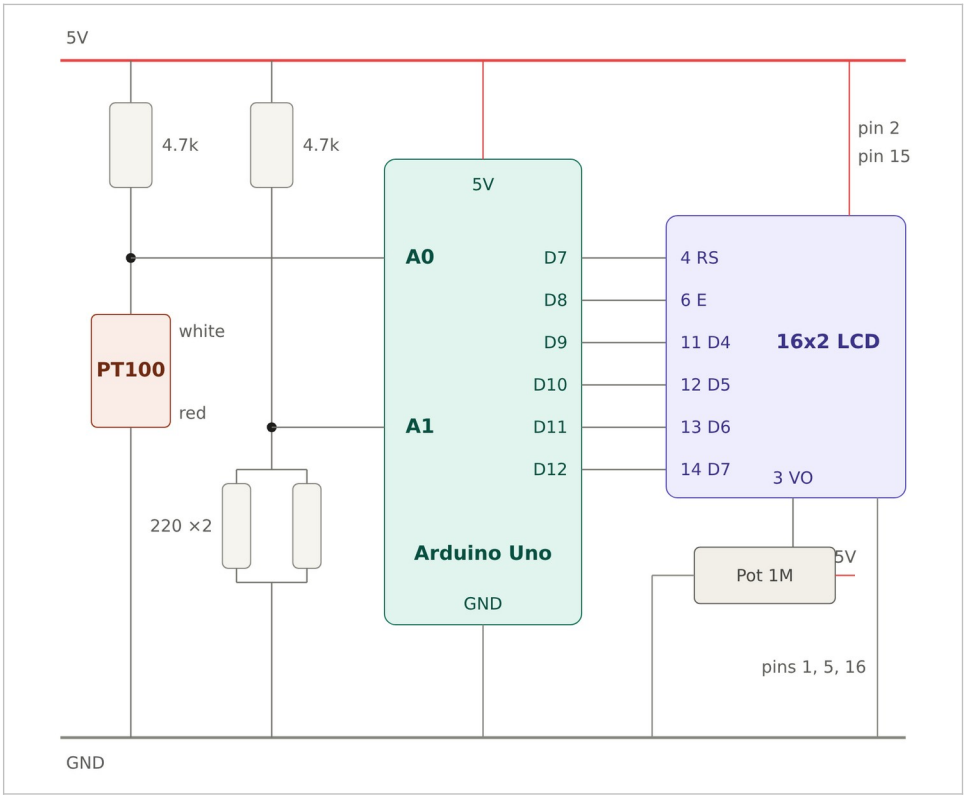


Figure 5. *New circuit with the two 4.7 kΩ resistors and the reference arm*

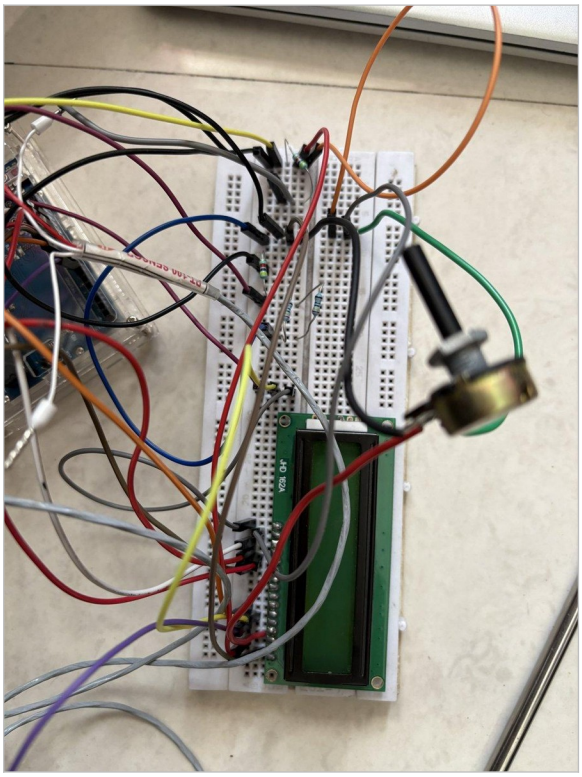


Figure 6. *Our breadboard circuit: Arduino, PT100 (labelled cable), LCD and contrast pot*

How this made it more accurate

- **Less self-heating.** The current through the PT100 dropped from about 2.2 mA to about **1 mA**, and the heat in the sensor from about 0.5 mW to about **0.1 mW**, about 4 times less.
- **The supply cancels out.** The A1 reference arm measures the real supply voltage in the same ADC units as A0: Vcc = V1 × (R_top2 + R_ref) / R_ref, then R_PT100 = R_top1 × V0 / (Vcc − V0). The reading no longer depends on the 5V rail being exact.
- **The trade-off:** the A0 signal became smaller (about 104 mV, down from about 240 mV), so each ADC step on the 5 V reference was worth about 12 °C. We fixed that later by switching to the 1.1 V reference (see “Code changes”).

04 Fixing the Readings

Still inaccurate: about 15 °C

With the new circuit the reading came down to about **15 °C (R = 106.0 Ω)**. That was much more reasonable, but still far too low for a room at around 28–30 °C.

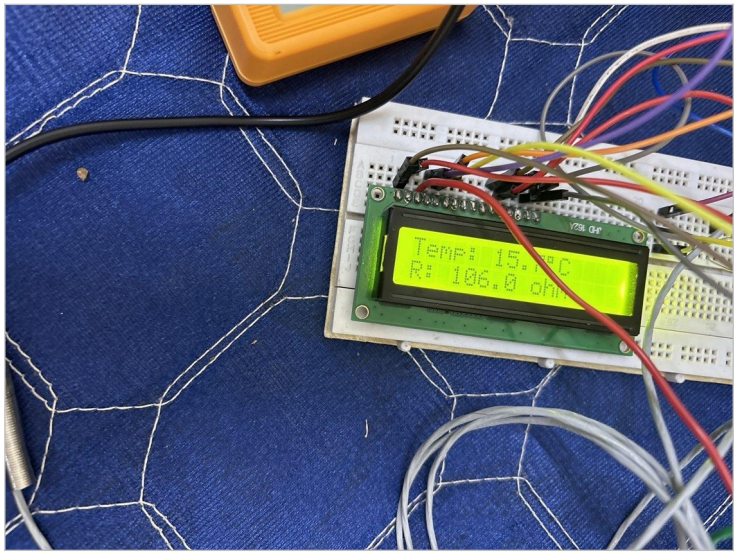


Figure 7. *After adding the 4.7 kΩ resistors: about 15 °C, still too low*

We found three more problems:

1. **The signal is tiny compared with the ADC step.** The A0 voltage is only about 104 mV and changes by about 0.39 mV per °C. On the default 5 V reference one ADC step is 4.9 mV, which is about **12 °C per step**. Being off by a single step explains a reading like this.
2. **We used the printed resistor values in the code instead of the real ones.** The "220 Ω" resistors actually measured 238 Ω and 237 Ω, so the reference was 118.75 Ω, not 110 Ω: an 8% error on everything calculated from A1. The 4.7 kΩ on the PT100 side was really 4650 Ω, adding about 1% (about 2.5 °C) more.
3. **Every part shared the same 5V and GND wires** as the LCD and its backlight. We didn't notice this until later (see "Showing low readings").

Changed the resistors

We measured every resistor separately with the multimeter and put the measured values into the code:

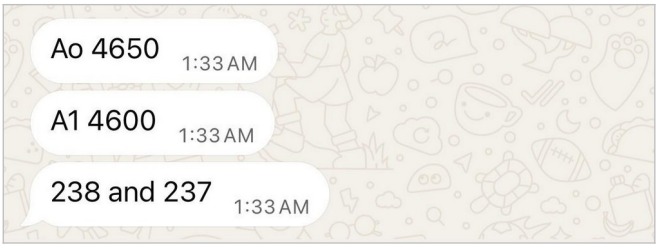


Figure 8. *Measured resistor values*

Resistor	Printed value	Measured value	Error
Top resistor, PT100 side (A0)	4.7 kΩ	4650 Ω	−1.1%
Top resistor, reference side (A1)	4.7 kΩ	4600 Ω	−2.1%
Reference resistor 1	220 Ω	238 Ω	+8.2%
Reference resistor 2	220 Ω	237 Ω	+7.7%
Reference pair in parallel	110 Ω	118.75 Ω	+8.0%

Code changes

- **1.1 V internal reference** (`analogReference(INTERNAL)`). The signals are well under 1.1 V, so each ADC step drops from about 12 °C to about **2.8 °C**.
- **Supply correction from A1.** Because the A1 arm measures the supply in the same ADC units as A0, the exact values of 5 V and 1.1 V cancel out.
- **Averaging 200 readings** per update to smooth out noise.
- **Callendar–Van Dusen equation** to convert resistance to temperature: $R = 100(1 + A{\cdot}T + B{\cdot}T^2)$, with $A = 3.9083\times10^{-3}$ and $B = -5.775\times10^{-7}$.
- A **"Check wiring"** message on the LCD when the readings are impossible.

Showing low readings

Even with the corrected values and new code, the readings stayed low: about **16.9 °C (R ≈ 106.6 Ω)**, and later **21 °C when the actual reading was about 30 °C**.

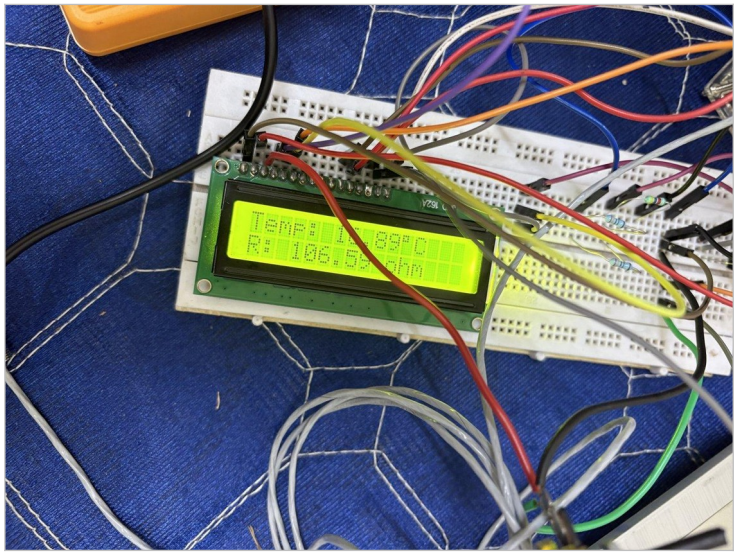


Figure 9. *Still low after the code changes, about 16.9 °C*

Checks we did

- **The code math was correct.** 106 Ω really does correspond to about 15 °C, so the conversion wasn't the problem. The measured resistance was.
- **Multimeter on the PT100:** 113.0 Ω. On its own that would mean about 33 °C, which is too warm for the room. But the meter also reads **1.5 Ω of probe/lead resistance**. Subtracting it gives 111.5 Ω, which is about **29.5 °C**: the room temperature, and almost exactly the 111.6 Ω the Arduino read once it was working.
- **Ratio check.** The circuit really measures the PT100 compared with the reference. By the meter that ratio is 111.5 / 118.75 = 0.94; by the Arduino it was about 0.89. A gap that size meant something in the live circuit was wrong, not the code or the sensor.

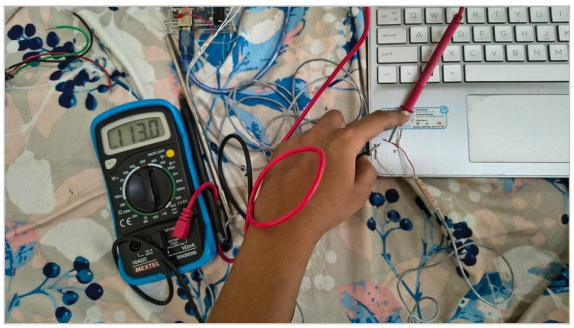


Figure 10. Multimeter across the PT100: 113.0 Ω

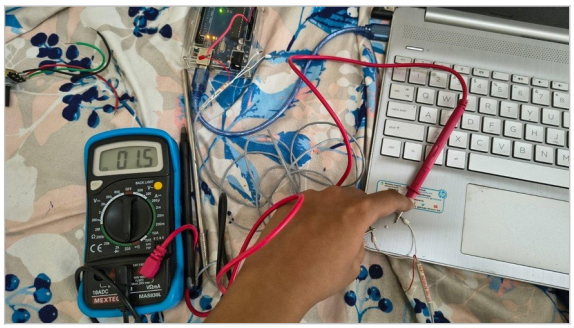


Figure 11. Probe/lead resistance: 1.5 Ω, to subtract

The reading jumped with the LCD backlight

At one point, with the LCD backlight off, the display showed **−33.71 °C (R = 86.76 Ω)**. A jump of about 20% from the backlight changing showed the real problem: the backlight's 20–100 mA current flowed through the same 5V and GND wires as the dividers. That shifts "ground" by a few millivolts, which is a large error on a signal of only about 110 mV.

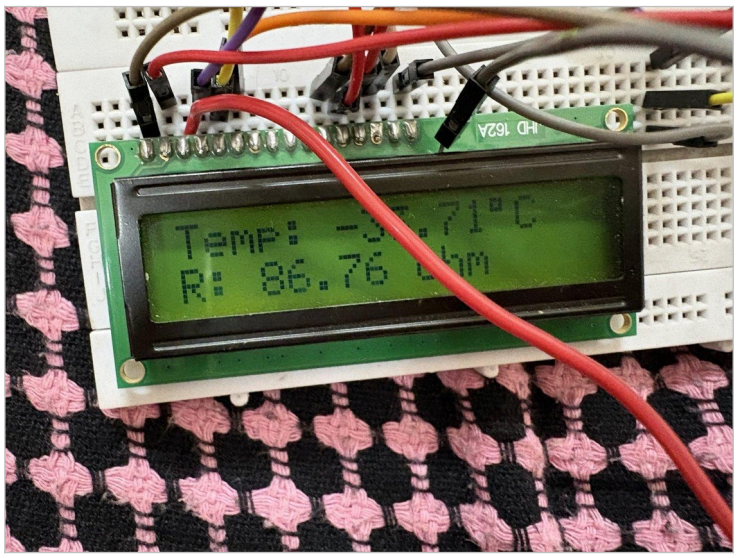


Figure 12. −33.71 °C with the backlight off, showing the shared-ground problem

Loose connections

The jumping readings also pointed to bad breadboard contacts. The likely spots were:

- Resistor legs not fully pushed into the breadboard (a bad contact of about 10 Ω in the reference arm makes the temperature read low).
- Stranded PT100 wires with loose strands touching the next row.
- The unconnected white PT100 wire touching the A0 row or ground.
- A split power rail in the middle of the breadboard.

Tightened the connections

- Re-seated every resistor leg in fresh breadboard holes.
- Twisted the PT100 wire strands tightly and taped off the spare white wire.
- Gave the dividers their own 5V and GND wires straight to the Arduino pins, and gave the LCD and potentiometer a separate GND pin.

- Cleared the old calibration (X in the Serial Monitor) and calibrated again, because the old points were taken with the faulty wiring.

Now working properly

The display then read **29.7–30.0 °C (R ≈ 111.6 Ω)**, matching the actual reading of about 30 °C.

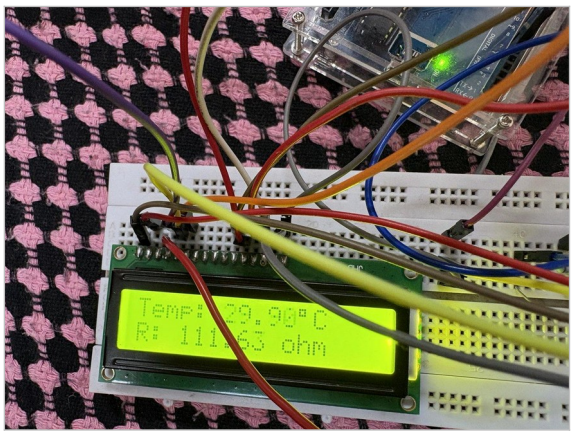


Figure 13. *Working: 29.90 °C, R = 111.63 Ω*

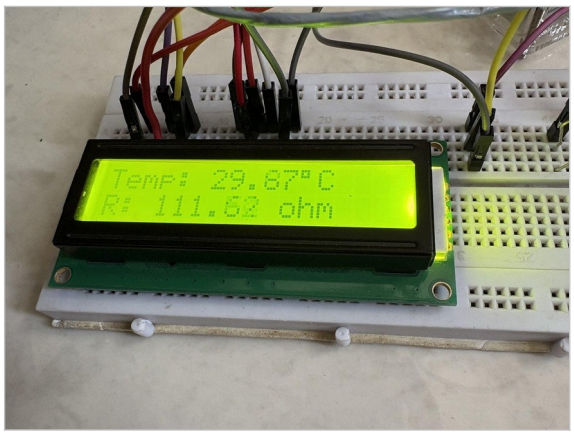


Figure 14. *Working: 29.87 °C, R = 111.62 Ω*

05 Calibrating with Least Squares

To correct the error that was left, we compared the Arduino's reading with the actual temperature at **11 points**: cold water, room temperature and hot water, plus eight readings of water in the kettle as it heated up.

Calibration readings

Point	Arduino reading	Actual temperature	Error
Cold water	14.48 °C	17.45 °C	−2.97 °C
Room	29.34 °C	32.45 °C	−3.11 °C
Hot water	46.53 °C	49.20 °C	−2.67 °C
Kettle 1	27.05 °C	28.50 °C	−1.45 °C
Kettle 2	29.70 °C	31.10 °C	−1.40 °C
Kettle 3	31.09 °C	33.20 °C	−2.11 °C
Kettle 4	33.11 °C	34.20 °C	−1.09 °C
Kettle 5	35.13 °C	37.40 °C	−2.27 °C
Kettle 6	27.15 °C	30.30 °C	−3.15 °C
Kettle 7	39.17 °C	42.40 °C	−3.23 °C
Kettle 8	35.30 °C	37.00 °C	−1.70 °C

The Arduino read low at every point: by about **3 °C** in cold, room and hot water, and by 1.1 to 3.2 °C in the kettle (2.3 °C on average across all 11 points).

Least-squares fit

We fitted a straight line, **actual = slope × reading + offset**, using the least-squares method: out of all possible straight lines, it is the one that makes the sum of the squared errors at our points as small as possible. With x as the Arduino reading, y as the actual temperature and n = 11 points:

$$\textbf{slope} = (n\Sigma xy - \Sigma x \cdot \Sigma y) / (n\Sigma x^2 - (\Sigma x)^2) = \mathbf{0.9954}$$

$$\textbf{offset} = \bar{y} - \text{slope} \times \bar{x} = 33.927 - 0.9954 \times 31.641 = \mathbf{2.432\ °C}$$

So the calibration is:

$$\textbf{actual temperature} = 0.9954 \times \textbf{Arduino reading} + 2.432\ °C$$

The slope is still almost exactly 1, so the error is mostly a constant offset rather than a scaling problem. Adding the kettle readings lowered the offset from 3.211 °C (our first fit, from the three points alone) to 2.432 °C, because the kettle readings were on average only about 2 °C low. Applied to the kettle readings, the first three-point calibration would have left errors of up to 1.8 °C, which is why we refitted using every point.

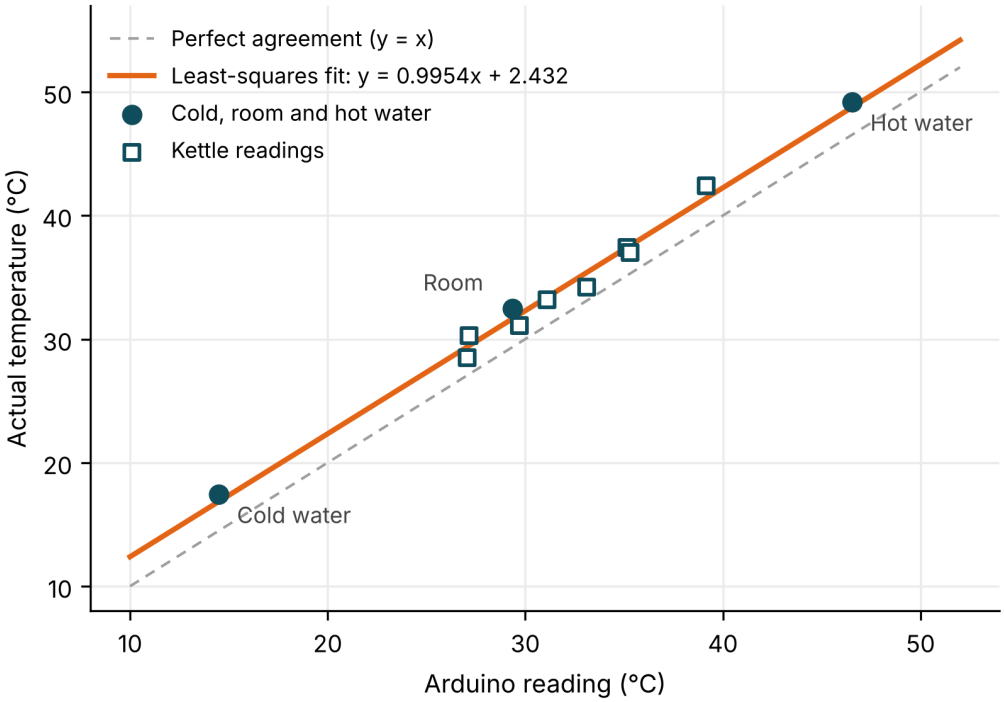


Figure 15. *Least-squares calibration line through all 11 points, compared with perfect agreement (y = x)*

After calibration

Point	Arduino reading	Calibrated	Actual	Error left
Cold water	14.48 °C	16.85 °C	17.45 °C	−0.60 °C
Room	29.34 °C	31.64 °C	32.45 °C	−0.81 °C
Hot water	46.53 °C	48.75 °C	49.20 °C	−0.45 °C
Kettle 1	27.05 °C	29.36 °C	28.50 °C	+0.86 °C
Kettle 2	29.70 °C	32.00 °C	31.10 °C	+0.90 °C
Kettle 3	31.09 °C	33.38 °C	33.20 °C	+0.18 °C
Kettle 4	33.11 °C	35.39 °C	34.20 °C	+1.19 °C
Kettle 5	35.13 °C	37.40 °C	37.40 °C	0.00 °C
Kettle 6	27.15 °C	29.46 °C	30.30 °C	−0.84 °C
Kettle 7	39.17 °C	41.42 °C	42.40 °C	−0.98 °C
Kettle 8	35.30 °C	37.57 °C	37.00 °C	+0.57 °C

After calibration every point is within **1.2 °C** of the actual temperature (RMS error 0.75 °C), compared with errors of up to 3.2 °C before. This is larger than the 0.13 °C we got from the first three points, but it is the more honest number: three points can only show how well a line fits those three points, while the kettle readings show how much our readings actually scatter between them.

Code change

We calculated the fit in Python and put the slope and offset into the sketch. The Arduino applies them to every reading, right after the Callendar–Van Dusen conversion, and shows the calibrated value on the LCD and Serial Monitor:

```
const float CAL_SLOPE  = 0.9954;
const float CAL_OFFSET = 2.4317;   // °C

float applyCalibration(float measuredC) {
  return CAL_SLOPE * measuredC + CAL_OFFSET;
}

// in loop(), after the temperature is calculated:
float calibrated = applyCalibration(temperature);
```

06 Mistakes We Made

Mistakes made during testing and their effects

1. Using a 2.2 kΩ resistor in the initial circuit

We initially used a 2.2 kΩ resistor with the PT100. Although the circuit was able to give readings, the voltage change for a change in temperature was relatively small. This reduced the effective resolution of the Arduino's ADC and made it harder to measure small temperature changes accurately.

2. Changing resistor values without checking the circuit calculations first

We considered using different resistor values to improve the readings, but we did not first establish the expected voltage for each configuration. This made it difficult to determine whether the changes were actually improving the circuit or introducing new errors.

3. Not checking the raw voltage before calculating temperature

We focused on the temperature displayed by the code instead of consistently verifying the ADC readings and corresponding voltage first. As a result, when the temperature changed from around 34 °C to 14 °C, we could not immediately determine whether the problem was in the circuit or the calculation.

4. Not implementing proper three-wire compensation for the PT100

We investigated the third wire after measuring the resistance between the sensor leads, but we did not implement a proper three-wire measurement circuit. Simply connecting the third wire would not compensate for lead resistance, so it would not automatically improve the accuracy.

5. Using an approximate temperature calculation without calibration

We used a simplified resistance-to-temperature equation to estimate the temperature. Since we had not calibrated the readings against a reference thermometer, we could not confirm how close the displayed temperature was to the actual temperature. This meant that the displayed value could not be treated as an accurate measurement.

6. Trying a bridge configuration without the external ADC used in the paper

We later experimented with measurements at A0 and A1, which gave ADC readings of approximately 23 and 25. These values were very low compared to the Arduino Uno's full ADC range of 0–1023, making small changes difficult to distinguish. The paper's high-precision bridge uses an ADS1115, which was unavailable for our setup, so our implementation did not have the same measurement capability.

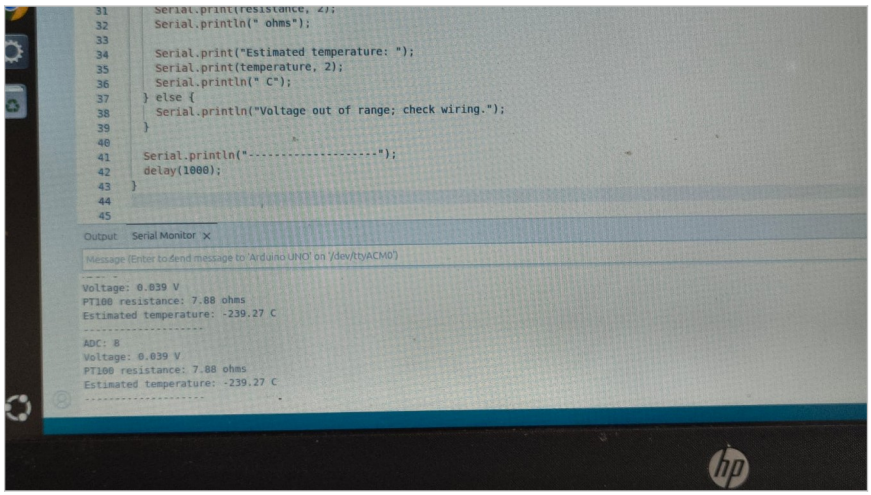


Figure 16. *Serial Monitor: ADC reading of 8 (0.039 V), giving 7.88 Ω and an estimated temperature of −239.27 °C*

7. Not testing each change separately

We changed the circuit and code during troubleshooting without consistently recording the readings before and after each change. This made it harder to identify the exact cause of the incorrect temperature readings and compare different configurations fairly.

07 Soldering the Perfboard



Figure 17. *Soldering the perfboard*

We soldered **two strips of female header pins** onto the perfboard for the connections to the Arduino (5V, GND, A0, A1 and D7–D12) and so that we wouldn't directly have to solder the pins of the LCD. All 5V points share one header pin and all GND points share another. That let us use ordinary **male-to-male jumper wires**: one end plugs into the header strip on the perfboard and the other into the Arduino's header. Nothing had to be cut, and any wire can be unplugged and swapped.

- The PT100 wires and all four resistors are soldered **directly** onto the perfboard, which gets rid of loose breadboard contacts.
- We used **insulated wire** anywhere two wires cross.

The layout we used:

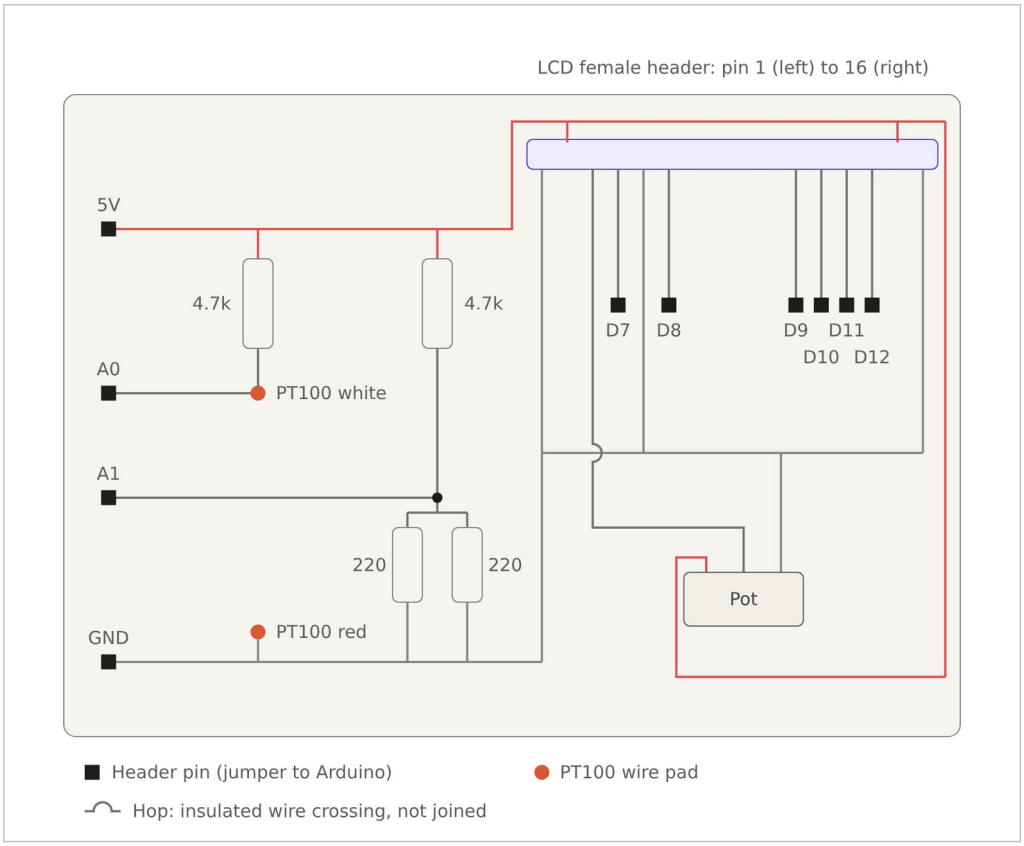


Figure 18. *Perfboard layout (top view)*

- we soldered all the 5V points (both 4.7 kΩ resistors, LCD pins 2 and 15, and the pot) together into one 5V line, and all the GND points (PT100 red wire, 220 Ω pair, LCD pins 1, 5 and 16, and the pot) into one GND line. Each line goes to the Arduino through a single header pin.
- **Left half (sensor):** the two dividers, with header pins for A0 and A1.
- **Right half (LCD):** a 16-pin female header so the LCD plugs in, with header pins for D7–D12.
- **Potentiometer:** left leg to 5V, middle leg to LCD pin 3, right leg to GND. Only the middle leg's position matters.
- Order: wire links first, then resistors, headers and pot, and the PT100 wires last.

08 3D Printing the Case

Same as the charger: perfboard and LCD on top of the Arduino, on a stand.

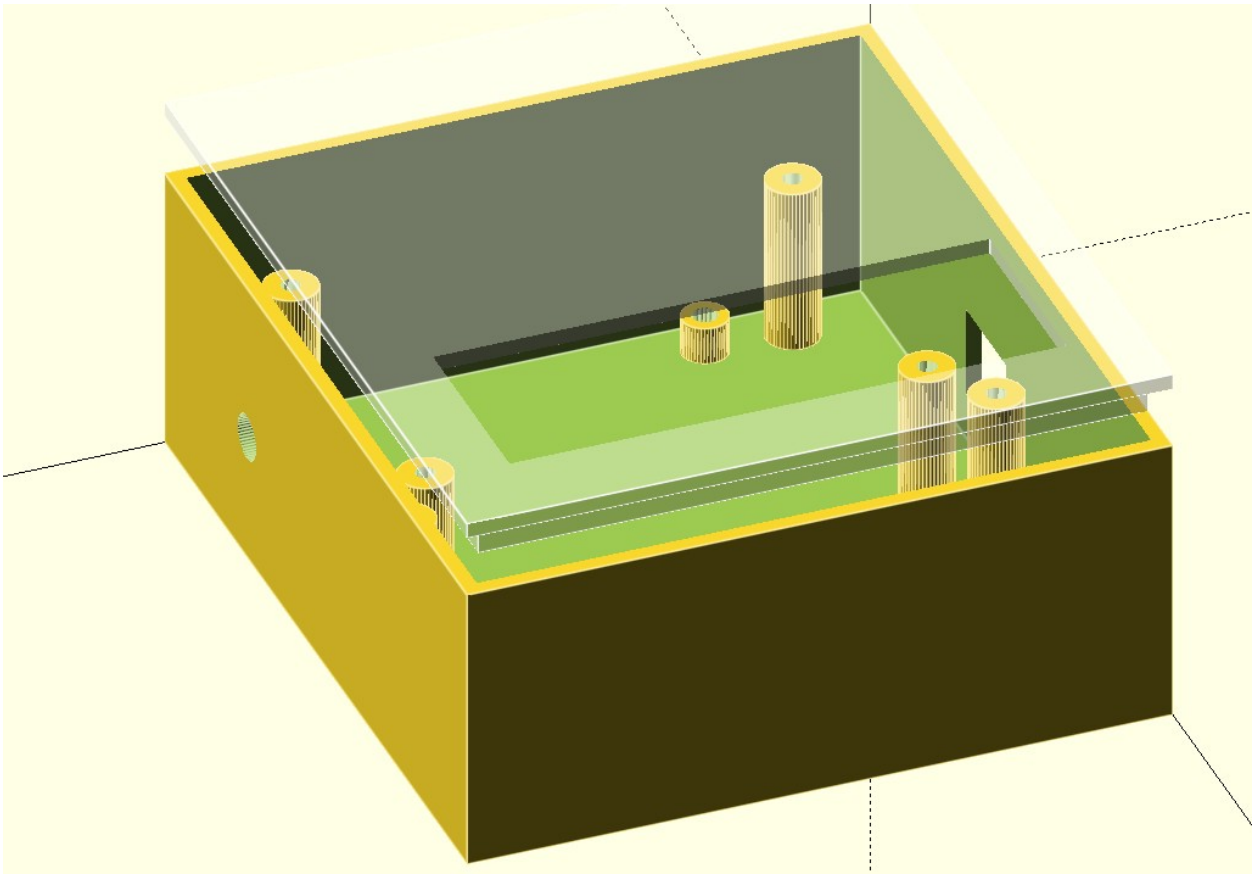


Figure 19. Our first case template, before we had real measurements

We modelled the case in **OpenSCAD**, where every size is a number at the top of the file. Changing a measurement rebuilds the whole case, so we could start from estimates and swap in real values as we measured them. The case prints in two parts: the **base** and the **lid**.

How the case is laid out

- **Bottom:** the Arduino Uno sits on the floor on four short standoffs, with its USB end facing the front wall.
- **Middle:** the perfboard sits on pillars above the Arduino.
- **Top:** the LCD is soldered onto the perfboard by its 16 pins. The pins sit in the perfboard's back row of holes along the long edge, with pin 1 in the first hole, so the LCD sticks out **31 mm** past the perfboard's back edge.
- **Lid:** a rectangular window over the LCD screen, and a lip that fits inside the base.
- **Holes:** one in the front wall for the USB cable, lined up with the Arduino's USB port, and one in the right wall, just above the perfboard, for the PT100 wires.
- **Overall size:** about **90 × 91 × 60 mm** with our current numbers.

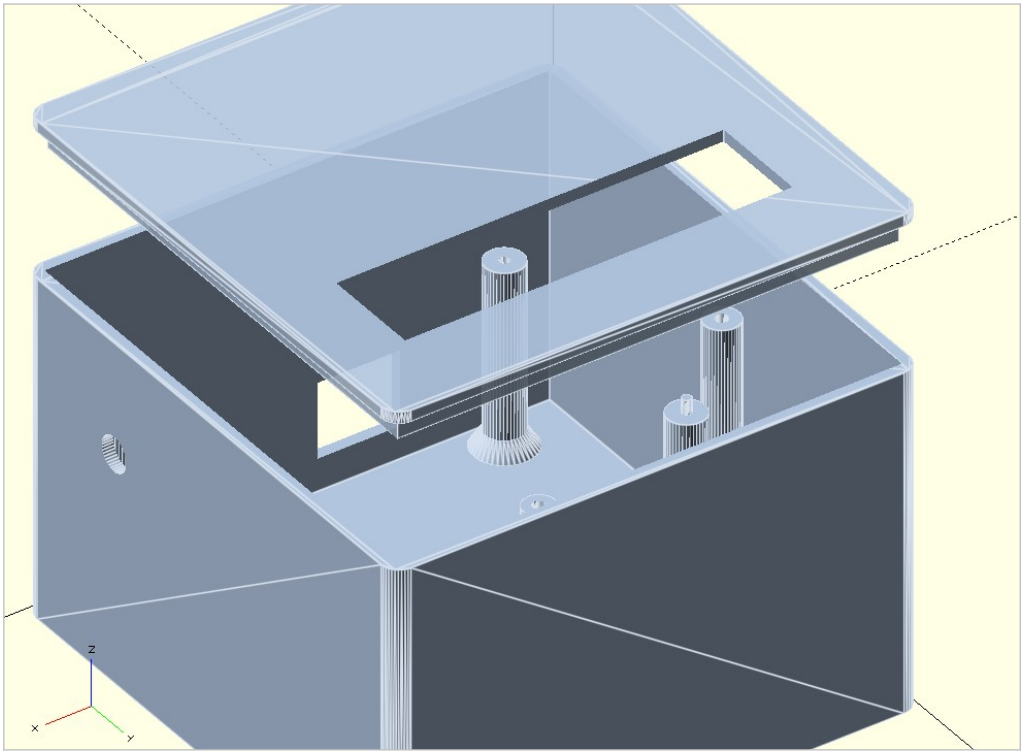


Figure 20. Current case model in OpenSCAD: base with the pillars and the USB and PT100 holes, lid with the LCD window

Measurements we used

Part	Value	Where it came from
Perfboard	50 × 70 mm, 2 mm corner holes about 2.5 mm in from the edges	Standard 5 × 7 cm board
LCD (JHD 162A)	80 × 36 mm board, four 2.7 mm holes on a 75 × 31 mm grid	Standard size and datasheet
LCD position	Pins in the perfboard's back row, pin 1 in the first hole; sticks out 31 mm past the back edge	Measured
Perfboard top to LCD board underside	12 mm	Measured
LCD black frame height	9 mm	Measured
Arduino	Official Uno: 68.6 × 53.3 mm, four mounting holes	Standard size
Power	USB cable	Our setup
Space above the Arduino for headers and wires	22 mm	-

How the height adds up

Part	Height
Floor	2 mm
Arduino standoffs	5 mm
Arduino board	1.6 mm
Space above the Arduino (assumed)	22 mm
Space under the perfboard for solder joints (assumed)	3 mm
Perfboard	1.6 mm

Part	Height
Perfboard to LCD board (measured)	12 mm
LCD board	1.6 mm
LCD black frame (measured)	9 mm
Air gap under the lid	0.5 mm
Lid	2 mm
Total	**60.3 mm**

The 22 mm above the Arduino is the biggest number. If we solder the wires instead of using plugged-in jumper wires, it drops to about 12 mm and the case becomes about 50 mm tall.

How each board is held

- **Arduino:** four solid standoffs at its mounting holes, each with a 2.2 mm pilot hole. M2.5 self-tapping screws go in from the top, so there are no holes through the floor and the bottom sits flat. A small **stop block** behind the Arduino's far edge stops it sliding back when we plug in the USB cable.
- **Perfboard:** the two front corners are screwed onto pillars with M2 self-tapping screws. The two back corners are under the LCD, where a screwdriver can't reach, so they drop onto pillars with small pins that fit the 2 mm holes.
- **LCD:** tall pillars at three of its four corner holes, with M2.5 self-tapping screws. The front-right hole is directly over the perfboard, so it has no pillar; the soldered pins hold that corner.
- **Screws needed:** 7 × M2.5 (3 for the LCD, 4 for the Arduino) and 2 × M2 for the perfboard, all self-tapping and 6–8 mm long.

Changes we made to the design

#	First version	What we changed
1	LCD drawn sitting on top of the perfboard	Moved it beside the perfboard, along the long side, as it is on our board
2	LCD position guessed	Placed it from where its pins are soldered: back row of holes, pin 1 in the first hole
3	Arduino held with screws and nuts through the floor	The nuts would stick out of the bottom and make the case wobble, and one Arduino hole is too close to the header for an M3 screw head. Switched to M2.5 self-tapping screws into solid standoffs
4	Nothing stopping the Arduino from sliding	Added a stop block behind it, because the USB plug pushes the board back every time
5	Perfboard screwed at all four corners	The back corners are under the LCD and can't be reached with a screwdriver, so they use pins instead